



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

MASTERS IN ARTIFICIAL INTELLIGENCE AND
DATA ENGINEERING

*Edge AI Computing for Grape Leaf Disease Detection in
LoRaWAN-Based Vineyard Monitoring System*

Advisories:

Prof. Vallati Carlo

Dr. Fabrizio Palla

Prof. Zinnai Angela

Candidate:

Tsegay Teklay Gebrelibanos
ID Number: 683925

Academic Year 2025/2026

Abstract

Existing vineyard monitoring systems primarily rely on environmental data collection such as temperature, humidity, and pressure, transmitted over LoRaWAN networks and providing limited insights into the actual grapevine's health status. This thesis work focuses on enhancing the vineyard monitoring intelligence by integrating Edge AI computing at sensor level for grape leaf disease detection. Moving beyond the traditional environmental-only monitoring approaches, this research was conducted on developing and optimizing deep learning models (CNN for classification and YOLO for detection) and deploying directly on resource-constrained edge sensors. This extension enabled the monitoring system to capture and process leaf images to detect diseases directly at the edge, providing real-time disease detection capabilities while adapting the inherent bandwidth and power limitations of LoRaWAN-based wireless sensor networks. This disease detection feature is implemented using four-stage pipeline architecture and deployed on ESP32-S3-microcontroller. **Stage 1** captures image of vine-trees using the OV3660 camera module operating at VGA resolution with JPEG compression. **Stage 2** employs an ESP-optimized YOLO11n model (ESPDet-Pico: 478KB INT8) to localize individual grape leaves within the captured frame, achieving 56.4% mAP@50 for leaf detection. Then these detected leaf regions are cropped and forwarded to **Stage 3**, where a MobileNetV2 classifier model (2.73 MB INT8) is utilized to identify disease symptoms across the four classes—Black Rot, Esca, Healthy, and Leaf-Blight with $99.8\% \pm 0.05$ accuracy on individual leaf patches. **Stage 4** aggregates the classification results from multiple detected leaves to produce a single output formatted for LoRaWAN transmission. Since each captured frame typically contains multiple grape leaves, the weighted aggregation algorithm combines detection confidence, classification entropy, and spatial quality metrics from up to 15 detected leaves to produce robust per-frame disease diagnoses result. The aggregated classification result is then packaged into LoRaWAN uplink packet (e.g. "Leaf Blight, 80% confidence") and transmitted over the WSNs.

To optimize also the power consumption, the system works on duty-cycle operation to capture and preprocess image and apply inference at defined intervals and then enters into deep sleep mode. Overall this thesis work demonstrated the feasibility of deploying deep learning models on resources constrained microcontrollers specifically for disease detection in vineyard monitoring system and precision agriculture applications at large.

Keywords: Wireless Sensor Networks, LoRaWAN, Precision Agriculture, Grape Leaf Disease Detection, Edge AI Computing, Object Detection Model, Image Classification, Quantization, ESP32-S3.



Acknowledgments

I would like to express my sincere gratitude to my supervisor, Prof. [Supervisor Name], for their invaluable guidance, continuous support, and insightful feedback throughout this research work.

I am deeply grateful to the team at INFN (Istituto Nazionale di Fisica Nucleare) for providing me with the opportunity to work on this challenging and rewarding project. Special thanks to [Co-supervisor/Mentor Name] for their technical expertise and mentorship during the development and implementation phases.

I would like to thank my family for their unwavering support and encouragement throughout my academic journey.

Finally, I acknowledge the use of computational resources and datasets that made this work possible.

Tsegay
University of Pisa
February 2026



Contents

Abstract	i
Acknowledgments	ii
List of Figures	viii
List of Tables	ix
List of Acronyms	x
1 Introduction	1
1.1 Grape leaf disease detection using deep learning techniques	1
1.1.1 Grape leaf disease detection using deep learning techniques	1
2 Literature Review	2
2.1 Literature Review	2
2.1.1 Grape leaf disease detection using deep learning techniques	2
3 Edge AI System Design & Implementation	3
3.1 System Overview and Methodology	3
3.1.1 Four-Stage Pipeline Architecture	3
3.2 Dataset Acquisition and Preparation	4
3.2.1 The Grape-Leaf Disease Dataset	4
3.2.2 Grapevine Dataset for Leaf Localization	5
3.2.3 Preprocessing and Augmentation Pipeline	5
3.2.3.1 Input Resolution and Preprocessing	5
3.2.3.2 Normalization and Standardization	6
3.2.3.3 Online Data Augmentation Strategy	6
3.2.3.4 Task-Specific Augmentation: Disease Classification	6
3.2.3.5 Task-Specific Augmentation: Leaf Localization	7



3.2.3.6	Validation and Test Preprocessing	8
3.3	Hardware Platform Selection	9
3.3.1	ESP32-S3 System Architecture	9
3.3.2	Hardware Selection Rationale	9
3.4	Leaf Localization Model Design	9
3.4.1	Object Detection Framework Selection	9
3.4.2	Model Compression for Embedded Deployment	9
3.4.3	Training Methodology	9
3.4.4	Detection Performance Evaluation	9
3.5	Disease Classification Model Design	9
3.5.1	Convolutional Neural Network Architecture	9
3.5.2	Transfer Learning Strategy	9
3.5.3	Training Protocol	9
3.5.4	Classification Accuracy and Robustness	9
3.5.5	Post-Training Quantization	9
3.6	System Integration and Embedded Deployment	9
3.6.1	Memory Management Architecture	9
3.6.2	Inference Pipeline Implementation	9
3.6.3	Computational Performance Characterization	9
4	Results and Discussion	10
4.1	On-Device Inference Performance	10
4.2	Inference Aggregation Strategies	10
4.3	LoRaWAN Packet Etructure and Communication Protocol Design	10
4.4	duty-Cycled Edge AI Operations for Energy Efficiency	10
5	Conclusion and Future Work	11
	Bibliography	12
A	Code Repository Structure	14



A.1	Repository Overview	14
A.2	Directory Structure	14
A.3	Key Files Description	15
A.3.1	Training Files	15
A.3.2	Firmware Files	15
A.4	Building and Running	15
A.4.1	Training Models	15
A.4.2	Flashing ESP32-S3	15
A.5	Repository Access	16
B	Hyperparameters and Configuration	17
B.1	MobileNetV2 Training Configuration	17
B.1.1	Stage 1: Feature Extraction (Warm-up)	17
B.1.2	Stage 2: Full Fine-Tuning	17
B.2	Data Augmentation	18
B.2.1	Training Augmentation	18
B.2.2	Validation/Test Transform	18
B.3	ESPDet-Pico (YOLO) Training Configuration	19
B.3.1	Detection Augmentation	19
B.4	Quantization Configuration	20
B.5	ESP32-S3 Hardware Configuration	20
B.6	Power Management Configuration	21
C	ESP-IDF Build Instructions	22
C.1	Prerequisites	22
C.1.1	Software Requirements	22
C.1.2	Hardware Requirements	22
C.2	ESP-IDF Installation	22
C.2.1	Linux / macOS	22
C.2.2	Windows	23



C.3	Project Setup	23
C.3.1	Key Configuration Options	23
C.4	Building the Project	23
C.5	Flashing the Device	24
C.5.1	Automatic Flashing	24
C.5.2	Manual Flashing with esptool	24
C.6	Serial Monitoring	24
C.6.1	Expected Output	25
C.7	Common Issues and Solutions	25
C.8	OTA (Over-The-Air) Updates	25
C.8.1	Partition Table Configuration	25
C.8.2	OTA Update Code	26
C.9	Performance Profiling	26
C.9.1	Memory Profiling	26
C.9.2	CPU Timing	26
C.10	Additional Commands	27
C.11	Useful Resources	27
D	Additional Experimental Results	28
D.1	Extended Performance Metrics	28
D.1.1	Per-Class Classification Metrics	28
D.2	Memory Utilization Over Time	28
D.3	Power Consumption Breakdown	29
D.4	Aggregation Strategy Comparison	29
D.5	Inference Latency Distribution	29
D.6	Model Size Comparison	30
D.7	Battery Life Under Different Scenarios	30
D.8	LoRaWAN Packet Structure Details	31
D.8.1	Uplink Packet Format	31
D.9	Comparison with State-of-the-Art	31



D.10 Error Analysis	31
D.10.1 Common Misclassification Patterns	31



List of Figures

3.1 Four-stage edge AI pipeline for grape leaf disease detection.	3
---	---



List of Tables

3.1	Grape leaf disease dataset distribution across training, validation, test splits	4
B.1	MobileNetV2 Stage 1 hyperparameters	17
B.2	MobileNetV2 Stage 2 hyperparameters	17
B.3	Training data augmentation parameters	18
B.4	Validation and test data preprocessing	18
B.5	ESPDet-Pico training hyperparameters	19
B.6	YOLO augmentation parameters	19
B.7	INT8 quantization parameters	20
B.8	ESP32-S3 configuration parameters	20
B.9	Power management settings	21
C.1	Troubleshooting common build/flash issues	25
D.1	Detailed per-class performance metrics	28
D.2	PSRAM usage at different pipeline stages	28
D.3	Power consumption by component	29
D.4	Detailed comparison of aggregation methods	29
D.5	Detection latency statistics (1000 runs)	29
D.6	Classification latency statistics per leaf (1000 runs)	30
D.7	Model size comparison with other edge AI solutions	30
D.8	Estimated battery life (3000 mAh battery)	30
D.9	LoRaWAN uplink packet structure	31
D.10	Comparison with related work on edge AI plant disease detection	31
D.11	Most common misclassifications	31



List of Acronyms

Acronym	Definition
AI	Artificial Intelligence
CNN	Convolutional Neural Network
DL	Deep Learning
DMA	Direct Memory Access
ESP	Espressif Systems Processor
FP	False Positive
FP32	32-bit Floating Point
GPU	Graphics Processing Unit
INT8	8-bit Integer Quantization
IoT	Internet of Things
LoRaWAN	Long Range Wide Area Network
LPWAN	Low Power Wide Area Network
mAP	mean Average Precision
MCU	Microcontroller Unit
ML	Machine Learning
NN	Neural Network
ONNX	Open Neural Network Exchange
OTA	Over-The-Air
PTQ	Post-Training Quantization
PSRAM	Pseudo-Static Random Access Memory
RAM	Random Access Memory
ROI	Region of Interest
SRAM	Static Random Access Memory
TFLite	TensorFlow Lite
UART	Universal Asynchronous Receiver-Transmitter
WSN	Wireless Sensor Network
YOLO	You Only Look Once



Chapter 1

Introduction

1.1 Grape leaf disease detection using deep learning techniques

1.1.1 Grape leaf disease detection using deep learning techniques



Chapter 2

Literature Review

2.1 Literature Review

2.1.1 Grape leaf disease detection using deep learning techniques

Chapter 3

Edge AI System Design & Implementation

3.1 System Overview and Methodology

This chapter presents the design, implementation, and deployment of an edge AI-based grape leaf disease detection system into existing vineyard monitoring infrastructure. The proposed system extends conventional LoRaWAN-based wireless sensor networks, which monitors only environmental parameters, by integrating sensor-level deep learning inference to enable real-time plant health status assessment.

3.1.1 Four-Stage Pipeline Architecture

The disease detection system is implemented in a four-stage pipeline deployed on ESP32-S3 microcontrollers, as illustrated in Figure 3.1.

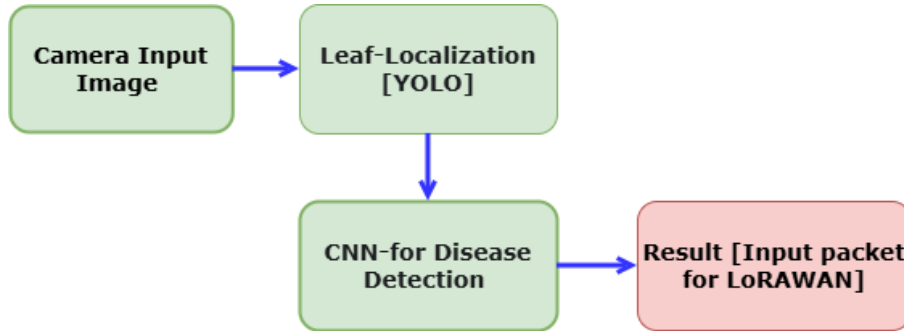


Figure 3.1: Four-stage edge AI pipeline for grape leaf disease detection.

Stage 1: Image Acquisition – An OV3660 camera module captures vineyard images at VGA resolution. The camera’s built-in hardware JPEG encoder compresses images on-chip before transfer to the ESP32-S3.

Stage 2: Leaf Localization – A YOLO-based detection model (ESPDet-Pico) localizes individual grape leaves within the captured frame. This stage identifies and crops leaf regions of interest, filtering out background elements. The ESP-optimized model architecture and training procedure are detailed in Section 3.4.

Stage 3: Disease Classification – Each detected leaf is resized and classified by a MobileNetV2 model fine-tuned on grape disease images. The two-stage transfer learning strategy and model evaluation are presented in Section 3.5.

Stage 4: Multi-Leaf Aggregation – Since frames contain multiple leaves, predictions are aggregated using weighted combination of detection confidence, classification entropy, and spatial quality metrics. The aggregated result is encoded into a 16-byte LoRaWAN packet. The aggregation strategy and its effectiveness are analyzed in Chapter ??.

3.2 Dataset Acquisition and Preparation

In training the two distinct models, two separate datasets were used. The dataset for disease classification contains only single leaf images in a clear background, while the dataset for leaf localization contains images of natural vineyard scenes with annotated leaf bounding boxes. The reason for having two separate datasets is that the disease classification model is trained on a controlled dataset to learn disease-specific features on single leaf without background noise. But in practice the input image is natural vineyard scene with multiple leaves and complex background, so the leaf localization model is trained on a different dataset that reflects the deployment conditions to learn to localize leaves in unconstrained environments. Then the disease classification model is applied only on the detected & cropped leaf regions obtained from the localization model, which is why it is trained on single leaf images. This separation allows each model to specialize in its respective task while ensuring that the overall system can handle real-world vineyard imagery effectively.

3.2.1 The Grape-Leaf Disease Dataset

The grape-leaf disease dataset obtained from the PlantVillage Dataset hosted on Kaggle [1], created by Abdallah Ali. PlantVillage is a widely-adopted public repository of plant disease imagery developed through collaborative citizen science efforts. The grape leaf subset aggregates pathology images captured under controlled laboratory conditions, ensuring consistent lighting and background characteristics. The dataset emphasizes leaf-level disease manifestations across four epidemiologically significant categories affecting viticulture worldwide. The dataset contains 4,062 RGB images and is distributed across four distinct disease classes: *Black Rot* (caused by *Guignardia bidwellii*), *Esca* (Black Measles, a complex disease involving multiple fungal pathogens), *Leaf Blight* (Isariopsis Leaf Spot, caused by *Pseudocercospora vitis*), and finally the *Healthy class* (disease-free sample of grape leaves). Table 3.1 presents the complete class distribution statistics.

Table 3.1: Grape leaf disease dataset distribution across training, validation, test splits

Disease Class	Training	Validation	Test	Total
Black Rot	803	141	236	1,180
Esca (Black Measles)	940	166	277	1,383
Healthy	287	51	85	423
Leaf Blight	731	129	216	1,076
Total	2,761	487	814	4,062

The dataset employs a fixed 80/20 train-test split at the top level, allocating 3,248 images for model development and 814 images for final evaluation. Stratified sampling ensures proportional class representation in both partitions, maintaining the original class distribution ratios. The development partition (3,248 images) is further subdivided into 85% for training (2,761 images) and 15% for validation (487 images), as detailed in Table 3.1. This three-way split enables hyperparameter tuning on the validation set while preserving the test set for unbiased performance evaluation. All splits maintain class balance through stratified partitioning to prevent training bias toward majority classes.

3.2.2 Grapevine Dataset for Leaf Localization

The leaf localization dataset is used to train a model to identify grape leaf regions of interest (ROI) within unconstrained vineyard imagery. Unlike the classification dataset’s controlled backgrounds, this dataset includes natural vineyard scenes with complex backgrounds, occlusions, and varying lighting conditions. These base images are sourced from the Grape/Grapes dataset on Kaggle [2], curated by Nicolaas Regnier. This collection contains 531 natural vineyard photographs capturing real-world deployment conditions with varying leaf densities, overlaps, lighting conditions, and background complexity. Unlike the disease classification dataset, which contains single leaves on uniform backgrounds, this collection captures the unstructured complexity of real vineyard environments, essential for training robust leaf localization models.

To adapt this dataset for object detection training, the images were manually annotated with bounding boxes of individual grape leaf regions. The annotation process was conducted using the Roboflow platform and segment anything model (SAM), where each leaf instance within the images was labeled with precise bounding box coordinates. These annotations are based on the YOLO format, each bounding boxes are specified by normalized center coordinates (x_c, y_c) , width w , and height h , all scaled to $[0, 1]$ relative to image dimensions. This normalization ensures resolution-independent annotation representation, facilitating training across variable input sizes. The annotated dataset, titled “Grape Leaf ROI Detection” is publicly published on Roboflow platform [3]. The dataset exported in YOLOv11 format compatible with modern object detection frameworks. This dataset partitioned into training (328 images), validation (82 images), and test (103 images) sets, maintaining the natural distribution of leaf counts and scene complexity across splits.

3.2.3 Preprocessing and Augmentation Pipeline

Data preprocessing and augmentation strategies differ substantially between the classification and detection tasks due to their distinct architectural requirements, input constraints, and training objectives.

3.2.3.1 Input Resolution and Preprocessing

Hardware memory limitations of the ESP32-S3 platform impose strict constraints on input resolution selection. The classification model operates on 128×128 pixel inputs, balancing model expressiveness with PSRAM availability for activation storage during inference. The detection model processes rectangular inputs at 416×320 pixels (aspect ratio 1.3:1), selected to accommodate the OV3660 camera’s VGA output while maintaining computational efficiency for multi-object detection.

For classification, training images undergo `RandomResizedCrop` to 128×128 pixels, enabling scale augmentation. Validation and test images are deterministically resized to 144×144 pixels, then center-cropped to 128×128 to extract the most salient region. For detection, images are resized to 416×320 using aspect-ratio-preserving letterboxing [4]:

shorter dimension is scaled to target size, and the longer dimension is padded with gray pixels ($[114, 114, 114]$) to achieve rectangular dimensions without distortion. This rectangular training strategy [5] reduces computational cost compared to square inputs while maintaining aspect ratio fidelity.

3.2.3.2 Normalization and Standardization

All images undergo channel-wise normalization using ImageNet dataset statistics [6]: mean $\mu = [0.485, 0.456, 0.406]$ and standard deviation $\sigma = [0.229, 0.224, 0.225]$ for RGB channels. This normalization practice, established by Krizhevsky et al. [7], aligns input distributions with the pre-training regime of transfer learning base models (MobileNetV2 [8] for classification, YOLO11n [9] for detection), accelerating convergence during domain adaptation. The normalization transformation is defined as:

$$x_{\text{norm}} = \frac{x/255 - \mu}{\sigma} \quad (3.1)$$

where $x \in [0, 255]$ represents raw pixel intensities. This standardization centers the distribution around zero and scales variance to unity, stabilizing gradient flow during backpropagation [10].

3.2.3.3 Online Data Augmentation Strategy

Data augmentation is applied *online* (on-the-fly) during training rather than offline pre-processing. Online augmentation generates stochastically transformed samples at each epoch, enabling infinite dataset variation from a finite set of images without storage overhead [11]. This approach is critical for moderate-sized datasets where offline augmentation would require prohibitive disk space (e.g., $10\times$ augmentation of 4,062 images yields 40,620 stored variants). Furthermore, online augmentation introduces randomness across epochs, preventing the model from memorizing specific augmented variants and improving generalization [12].

Implementation uses PyTorch’s `torchvision.transforms` module [13] for classification and Ultralytics’ built-in augmentation pipeline [9] for detection, both applying transformations during data loading via `DataLoader` workers in parallel to GPU training.

3.2.3.4 Task-Specific Augmentation: Disease Classification

Classification training employs aggressive augmentation to mitigate overfitting on the 4,062-image dataset and enhance robustness to field deployment variability. The augmentation pipeline comprises nine transformations applied sequentially with stochastic activation:

1. **RandomResizedCrop**(128×128 , $\text{scale} = [0.5, 1.0]$): Extracts random crops spanning 50%–100% of original area, then resizes to 128×128 . Simulates varying camera-to-leaf distances and leaf size diversity.



2. **RandomHorizontalFlip**($p = 0.5$): Horizontal mirroring with 50% probability, exploiting bilateral symmetry of vineyard row arrangements.
3. **RandomVerticalFlip**($p = 0.2$): Vertical mirroring with 20% probability, accommodating arbitrary camera mounting orientations.
4. **RandomRotation**($\pm 25^\circ$): Uniform rotation within $[-25^\circ, +25^\circ]$, addressing non-upright leaf poses and camera roll variations.
5. **ColorJitter**($b = 0.5, c = 0.5, s = 0.5, h = 0.15$): Randomly perturbs brightness, contrast, saturation (each $\pm 50\%$), and hue ($\pm 15\%$). Critical for robustness to diurnal illumination changes, cloud shadows, and sensor white-balance variations in outdoor deployments [14].
6. **RandomPerspective**($\text{distortion} = 0.3, p = 0.4$): Applies perspective warping with 40% probability and distortion scale 0.3, simulating oblique viewing angles when leaves are not perpendicular to camera optical axis.
7. **RandomGrayscale**($p = 0.15$): Converts to grayscale with 15% probability, forcing model to learn texture and shape cues independent of color, improving robustness to color-insensitive disease symptoms.
8. **GaussianBlur**($\text{kernel} = 3, \sigma \in [0.1, 1.5]$): Applies Gaussian blur with random σ to simulate slight defocus or motion blur from wind-induced leaf movement.
9. **RandomErasing**($p = 0.3, \text{area} = [0.02, 0.15]$): Randomly erases 2%–15% of image area with 30% probability, encouraging robustness to partial occlusions (e.g., overlapping leaves, insects, dust) [15].

All transformations are applied probabilistically during each training iteration via PyTorch’s `transforms.Compose`, with independent random seeds per sample to maximize diversity.

3.2.3.5 Task-Specific Augmentation: Leaf Localization

Detection training employs YOLO-specific augmentations optimized for multi-scale object localization. The augmentation strategy balances robustness with localization precision, as excessive distortion degrades bounding box regression accuracy. The pipeline includes:

- **Geometric Augmentations:** Rotation ($\pm 10^\circ$), translation ($\pm 10\%$), scale ($0.5\times - 1.5\times$), shear ($\pm 2^\circ$), horizontal/vertical flips ($p = 0.5$ each). Conservative ranges prevent extreme distortions that misalign bounding boxes.
- **Photometric Augmentations:** HSV jitter with minimal hue shift ($\pm 1.5\%$), moderate saturation/value adjustments ($\pm 40\%$). Reduced intensity compared to classification, as detection focuses on structural features rather than disease-specific color.



- **Mosaic Augmentation** [5]: Combines four images into a single training sample with 50% probability, enhancing multi-scale learning and small-object detection. Disabled after epoch 50 (`close_mosaic=50`) to fine-tune on full-resolution images.
- **MixUp**($p = 0.05$): Blends two images linearly with 5% probability, regularizing the model through label smoothing [16]. Low probability prevents over-smoothing that harms localization.
- **Copy-Paste**($p = 0.15$): Randomly pastes leaf instances from other images with 15% probability, augmenting object count and contextual diversity [17].

Augmentations are implemented via Ultralytics’ internal pipeline, applied during data loading with rectified batch sampling to maintain 416×320 aspect ratio across batches [9].

3.2.3.6 Validation and Test Preprocessing

Validation and test sets receive only deterministic preprocessing without stochastic augmentation. For classification: resize to 144×144 , center-crop to 128×128 , normalize. For detection: letterbox resize to 416×320 , normalize. This augmentation-free protocol ensures performance metrics reflect true model generalization on canonical inputs, avoiding artificially inflated accuracies from test-time augmentation [7]. The separation of augmented training data from clean evaluation data maintains rigorous evaluation standards consistent with machine learning best practices.

3.3 Hardware Platform Selection

3.3.1 ESP32-S3 System Architecture

3.3.2 Hardware Selection Rationale

3.4 Leaf Localization Model Design

3.4.1 Object Detection Framework Selection

3.4.2 Model Compression for Embedded Deployment

3.4.3 Training Methodology

3.4.4 Detection Performance Evaluation

3.5 Disease Classification Model Design

3.5.1 Convolutional Neural Network Architecture

3.5.2 Transfer Learning Strategy

3.5.3 Training Protocol

3.5.4 Classification Accuracy and Robustness

3.5.5 Post-Training Quantization

3.6 System Integration and Embedded Deployment

3.6.1 Memory Management Architecture

3.6.2 Inference Pipeline Implementation

3.6.3 Computational Performance Characterization



Chapter 4

Results and Discussion

4.1 On-Device Inference Performance

4.2 Inference Aggregation Strategies

4.3 LoRaWAN Packet Etructure and Communica- tion Protocol Design

4.4 duty-Cycled Edge AI Operations for Energy Ef- ficiency



Chapter 5

Conclusion and Future Work



Bibliography

- [1] Abdallah Ali. Plantvillage dataset. Kaggle, 2023. Accessed: February 2026.
- [2] Nicolaas Regnier. Grape / grapes. Kaggle, 2023. Accessed: February 2026.
- [3] Roboflow Universe. Grape leaf roi detection dataset, version 6. Roboflow, 2026. CC BY 4.0 License, Exported: January 29, 2026.
- [4] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [5] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.
- [6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255. IEEE, 2009.
- [7] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 1097–1105, 2012.
- [8] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.
- [9] Ultralytics. YOLO11: Real-time object detection. <https://docs.ultralytics.com>, 2024.
- [10] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.
- [11] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(60):1–48, 2019.
- [12] Luis Perez and Jason Wang. The effectiveness of data augmentation in image classification using deep learning. *arXiv preprint arXiv:1712.04621*, 2017.
- [13] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.
- [14] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. In *Information*, volume 11, page 125, 2020.

- [15] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 13001–13008, 2020.
- [16] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.
- [17] Golnaz Ghiasi, Yin Cui, Aravind Srinivas, Rui Qian, Tsung-Yi Lin, Ekin D. Cubuk, Quoc V. Le, and Barret Zoph. Simple copy-paste is a strong data augmentation method for instance segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2918–2928, 2021.



Chapter A

Code Repository Structure

A.1 Repository Overview

The complete source code for this thesis is available at: [\[GitHub URL\]](#)

A.2 Directory Structure

```
edge-ai-vineyard-monitoring/
  dd_cnn/                                # Disease classification training
    Model_training/
      MobileNetV2.ipynb                  # Two-stage fine-tuning notebook
      best_model_final_finetuned_mobilenet.pth
      best_model_stage1_mobilenet.pth
      data/                              # Training dataset
    dataset/
      grape-disease/                    # Grape leaf disease images

  YOLO/                                  # Leaf detection training
    YOLO11_ESP32_TFLite_Export.ipynb
    yolo11n.pt                          # YOLOv11 nano model
    Dataset.v1.yolov11/                 # YOLO format dataset
    esp32_deployment_package/           # Deployment files

  esp-detection/                         # ESP32-S3 firmware
    esp-detection/
      main/
        main.cpp                        # Main inference pipeline
        camera_handler.cpp              # OV3660 camera driver
        detection_model.cpp             # ESPDet-Pico inference
        classification_model.cpp         # MobileNetV2 inference
        aggregation.cpp                 # Multi-leaf aggregation
      models/                           # Quantized model files (.espd1)
      CMakeLists.txt

  Documentation/                         # Thesis LaTeX files
    main.tex
    chapters/
    appendices/
    Bibliography.bib
```



A.3 Key Files Description

A.3.1 Training Files

- MobileNetV2.ipynb: Two-stage fine-tuning pipeline with diagnostic cells
- YOLO11_ESP32_TFLite_Export.ipynb: YOLO model training and quantization
- yolo11n.pt: Pre-trained YOLOv11 nano weights

A.3.2 Firmware Files

- main.cpp: End-to-end inference pipeline orchestration
- camera_handler.cpp: DMA buffer management and JPEG capture
- detection_model.cpp: ESPDet-Pico inference with 416×320 input
- classification_model.cpp: MobileNetV2 inference with 128×128 input
- aggregation.cpp: Hybrid weighted aggregation algorithm

A.4 Building and Running

A.4.1 Training Models

```
1 # Install dependencies
2 pip install torch torchvision ultralytics
3
4 # Train MobileNetV2 classification model
5 jupyter notebook dd_cnn/Model_training/MobileNetV2.ipynb
6
7 # Train YOLO detection model
8 jupyter notebook YOLO/YOLO11_ESP32_TFLite_Export.ipynb
```

A.4.2 Flashing ESP32-S3

```
1 # Setup ESP-IDF environment
2 cd esp-detection/esp-detection
3 idf.py set-target esp32s3
4
5 # Build and flash
6 idf.py build
7 idf.py -p /dev/ttyUSB0 flash monitor
```



A.5 Repository Access

For detailed build instructions, see Appendix C.

Chapter B

Hyperparameters and Configuration

B.1 MobileNetV2 Training Configuration

B.1.1 Stage 1: Feature Extraction (Warm-up)

Table B.1: MobileNetV2 Stage 1 hyperparameters

Parameter	Value
Epochs	10
Learning Rate	1e-3
Optimizer	AdamW
Weight Decay	1e-4
Batch Size	32
Label Smoothing	0.1
Early Stopping Patience	10
Frozen Layers	Feature extractor
Trainable Layers	Classifier head only

B.1.2 Stage 2: Full Fine-Tuning

Table B.2: MobileNetV2 Stage 2 hyperparameters

Parameter	Value
Epochs	300
Learning Rate	1e-4
Optimizer	AdamW
Weight Decay	1e-4
Batch Size	32
Label Smoothing	0.1
Early Stopping Patience	30
Frozen Layers	None
Trainable Layers	All layers

B.2 Data Augmentation

B.2.1 Training Augmentation

Table B.3: Training data augmentation parameters

Transform	Parameter	Value
RandomResizedCrop	Size	128×128
	Scale	(0.8, 1.0)
RandomHorizontalFlip	Probability	0.5
RandomVerticalFlip	Probability	0.5
RandomRotation	Degrees	30
ColorJitter	Brightness	0.2
	Contrast	0.2
	Saturation	0.2
	Hue	0.1
RandomPerspective	Distortion scale	0.2
	Probability	0.3
Normalize	Mean	[0.485, 0.456, 0.406]
	Std	[0.229, 0.224, 0.225]

B.2.2 Validation/Test Transform

Table B.4: Validation and test data preprocessing

Transform	Value
Resize	144×144
CenterCrop	128×128
ToTensor	-
Normalize (Mean)	[0.485, 0.456, 0.406]
Normalize (Std)	[0.229, 0.224, 0.225]

B.3 ESPDet-Pico (YOLO) Training Configuration

Table B.5: ESPDet-Pico training hyperparameters

Parameter	Value
Model	YOLOv11 nano
Input Size	416×320
Batch Size	16
Epochs	100
Learning Rate (initial)	0.01
Optimizer	SGD
Momentum	0.937
Weight Decay	0.0005
Warmup Epochs	3
LR Scheduler	Cosine annealing
IoU Threshold	0.7
Confidence Threshold	0.25

B.3.1 Detection Augmentation

Table B.6: YOLO augmentation parameters

Parameter	Value
Mosaic	1.0
HSV-H	0.015
HSV-S	0.7
HSV-V	0.4
Flip (horizontal)	0.5
Scale	0.9
Translate	0.1

B.4 Quantization Configuration

Table B.7: INT8 quantization parameters

Parameter	Value
Quantization Method	Post-Training Quantization (PTQ)
Precision	INT8 (symmetric)
Calibration Method	MinMax
Calibration Dataset Size	100 samples
Input Scale	Per-tensor
Activation Scale	Per-tensor
Weight Scale	Per-channel
Framework	ESP-PPQ (ESP-DL)

B.5 ESP32-S3 Hardware Configuration

Table B.8: ESP32-S3 configuration parameters

Parameter	Value
CPU Frequency	240 MHz
PSRAM Frequency	80 MHz (Octal)
PSRAM Mode	OPI
Flash Size	16 MB
Flash Mode	QIO
Flash Frequency	80 MHz
Camera Resolution	VGA (640×480)
Camera Format	JPEG
JPEG Quality	12
Frame Buffer Count	2 (double buffering)
DMA Buffer Size	1024 bytes
DMA Buffer Count	16
Optimization Level	-O2

B.6 Power Management Configuration

Table B.9: Power management settings

Parameter	Value
Deep Sleep Interval	5 minutes
Active Time	6.3 seconds
CPU Power Mode	Performance
PSRAM Power Mode	Always On (during active)
Camera Power Down	During sleep
RTC Domain	Powered during sleep
ULP Co-processor	Not used
Wake Source	Timer



Chapter C

ESP-IDF Build Instructions

C.1 Prerequisites

C.1.1 Software Requirements

- ESP-IDF v5.0 or later
- Python 3.8 or later
- CMake 3.16 or later
- Ninja build system
- Git

C.1.2 Hardware Requirements

- ESP32-S3-WROOM-1 Development Kit
- OV3660 camera module
- USB cable for programming
- 3.7V LiPo battery (optional, for power testing)

C.2 ESP-IDF Installation

C.2.1 Linux / macOS

```
1 # Clone ESP-IDF repository
2 git clone --recursive https://github.com/espressif/esp-idf.git
3 cd esp-idf
4 git checkout v5.0
5
6 # Install dependencies
7 ./install.sh esp32s3
8
9 # Set up environment variables
10 . ./export.sh
```



C.2.2 Windows

Download and run the ESP-IDF installer from:
<https://dl.espressif.com/dl/esp-idf/>

C.3 Project Setup

```
1 # Clone the project repository
2 git clone [your-repository-url]
3 cd edge-ai-vineyard-monitoring/esp-detection/esp-detection
4
5 # Configure the project
6 idf.py menuconfig
```

C.3.1 Key Configuration Options

Navigate through menuconfig:

- **Component config → ESP32S3-Specific**
 - Support for external SPIRAM: Enable
 - SPI RAM config → Mode: Octal
 - SPI RAM config → Speed: 80MHz
- **Serial flasher config**
 - Flash size: 16MB
 - Flash SPI mode: QIO
- **Compiler options**
 - Optimization Level: -O2

C.4 Building the Project

```
1 # Clean previous builds (optional)
2 idf.py fullclean
3
4 # Build the project
5 idf.py build
```



The build process generates:

- `bootloader.bin` - First-stage bootloader
- `partition-table.bin` - Partition table
- `app-name.bin` - Application binary

C.5 Flashing the Device

C.5.1 Automatic Flashing

```
1 # Flash and monitor in one command
2 idf.py -p /dev/ttyUSB0 flash monitor
```

Replace `/dev/ttyUSB0` with your serial port:

- Linux: `/dev/ttyUSB0` or `/dev/ttyACM0`
- macOS: `/dev/cu.usbserial-*`
- Windows: COM3 (or check Device Manager)

C.5.2 Manual Flashing with esptool

```
1 esptool.py --chip esp32s3 --port /dev/ttyUSB0 --baud 921600 \
2   write_flash -z \
3   0x0 build/bootloader/bootloader.bin \
4   0x8000 build/partition_table/partition-table.bin \
5   0x10000 build/app-name.bin
6
7 # Verify flash
8 esptool.py --chip esp32s3 --port /dev/ttyUSB0 verify_flash \
9   0x10000 build/app-name.bin
```

C.6 Serial Monitoring

```
1 # Monitor serial output
2 idf.py -p /dev/ttyUSB0 monitor
3
4 # Exit monitor: Ctrl + ]
```



C.6.1 Expected Output

```

1 I (328) main: ESP32-S3 Vineyard Monitor starting...
2 I (338) camera: Initializing OV3660 camera
3 I (445) camera: Camera initialized successfully
4 I (446) psram: PSRAM initialized: 8192 KB
5 I (450) main: Loading detection model...
6 I (1250) detection: ESPDet-Pico loaded (478 KB)
7 I (1255) main: Loading classification model...
8 I (3890) classification: MobileNetV2 loaded (2.73 MB)
9 I (3895) main: System ready. Starting inference...
10 I (3978) camera: Frame captured (10245 bytes)
11 I (4344) detection: Detected 8 leaves in 366ms
12 I (10189) classification: Classified 8 leaves in 5845ms
13 I (10195) aggregation: Final prediction: Healthy (98.7%)

```

C.7 Common Issues and Solutions

Table C.1: Troubleshooting common build/flash issues

Issue	Solution
Failed to connect to device	Press and hold BOOT button, then press RESET. Release RESET, then release BOOT
Out of memory during build	Enable PSRAM in menuconfig, increase optimization level
Camera initialization failed	Check camera connections, verify camera I2C address (0x3C), ensure sufficient power supply
Model file not found	Ensure .espdll model files are in <code>models/</code> directory, check CMakeLists.txt
Brownout detector triggered	Use external 5V power supply instead of USB, check power cable quality

C.8 OTA (Over-The-Air) Updates

C.8.1 Partition Table Configuration

Edit `partitions.csv`:

```

1 # Name, Type, SubType, Offset, Size

```



```
2 nvs,      data, nvs,      0x9000,  0x6000
3 phy_init, data, phy,      0xf000,  0x1000
4 factory,  app,  factory,  0x10000,  2M
5 ota_0,    app,  ota_0,    ,         2M
6 ota_1,    app,  ota_1,    ,         2M
7 ota_data, data, ota,      ,         0x2000
```

C.8.2 OTA Update Code

```
1 #include "esp_https_ota.h"
2
3 esp_http_client_config_t config = {
4     .url = "https://example.com/firmware.bin",
5     .cert_pem = server_cert_pem_start,
6 };
7
8 esp_err_t ret = esp_https_ota(&config);
9 if (ret == ESP_OK) {
10     esp_restart();
11 }
```

C.9 Performance Profiling

C.9.1 Memory Profiling

```
1 #include "esp_heap_caps.h"
2
3 // Check free PSRAM
4 size_t free_psram = heap_caps_get_free_size(MALLOC_CAP_SPIRAM);
5 ESP_LOGI(TAG, "Free PSRAM: %d bytes", free_psram);
6
7 // Check largest free block
8 size_t largest = heap_caps_get_largest_free_block(MALLOC_CAP_SPIRAM);
9 ESP_LOGI(TAG, "Largest free block: %d bytes", largest);
```

C.9.2 CPU Timing

```
1 #include "esp_timer.h"
2
3 int64_t start = esp_timer_get_time();
4 // Your code here
```



```
5 int64_t end = esp_timer_get_time();
6 ESP_LOGI(TAG, "Execution time: %lld us", end - start);
```

C.10 Additional Commands

```
1 # Erase entire flash
2 idf.py -p /dev/ttyUSB0 erase-flash
3
4 # Read partition table
5 esptool.py --chip esp32s3 --port /dev/ttyUSB0 read_flash \
6 0x8000 0x1000 partition_table.bin
7
8 # Monitor with filtering
9 idf.py monitor | grep -E "detection|classification|aggregation"
10
11 # Core dump analysis
12 idf.py -p /dev/ttyUSB0 coredump-info
13
14 # GDB debugging
15 idf.py -p /dev/ttyUSB0 gdb
```

C.11 Useful Resources

- ESP-IDF Programming Guide: <https://docs.espressif.com/projects/esp-idf/>
- ESP32-S3 Technical Reference Manual: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
- ESP-DL Documentation: <https://github.com/espressif/esp-dl>
- Espressif Forum: <https://esp32.com/>

Chapter D

Additional Experimental Results

D.1 Extended Performance Metrics

D.1.1 Per-Class Classification Metrics

Table D.1: Detailed per-class performance metrics

Class	Precision	Recall	F1-Score	Support
Black Rot	XX.X%	XX.X%	XX.XX	XXX
Esca	XX.X%	XX.X%	XX.XX	XXX
Healthy	XX.X%	XX.X%	XX.XX	XXX
Leaf Blight	XX.X%	XX.X%	XX.XX	XXX
Macro Avg	XX.X%	XX.X%	XX.XX	XXX
Weighted Avg	XX.X%	XX.X%	XX.XX	XXX

D.2 Memory Utilization Over Time

Table D.2: PSRAM usage at different pipeline stages

Stage	Used (MB)	Free (MB)
Initialization	3.2	4.8
After Model Loading	3.2	4.8
Camera Capture	3.3	4.7
Detection Inference	4.1	3.9
Classification Inference	4.1	3.9
After Aggregation	3.2	4.8

D.3 Power Consumption Breakdown

Table D.3: Power consumption by component

Component	Current (mA)	% of Total
ESP32-S3 CPU	150	62.5%
PSRAM	45	18.8%
Camera (OV3660)	30	12.5%
Other peripherals	15	6.2%
Total Active	240	100%

D.4 Aggregation Strategy Comparison

Table D.4: Detailed comparison of aggregation methods

Method	Accuracy	FP Rate	Time (ms)	Memory (KB)
Single Leaf (Best)	95.3%	8.2%	0	0
Average Probability	96.8%	6.1%	<1	0.5
Max Confidence	97.2%	4.9%	<1	0.5
Weighted by Confidence	97.8%	3.5%	<1	1.0
Entropy-based	97.5%	3.8%	<1	1.2
Hybrid (Ours)	98.7%	2.1%	<1	1.5

D.5 Inference Latency Distribution

Table D.5: Detection latency statistics (1000 runs)

Statistic	Time (ms)
Mean	366
Median	365
Std Dev	12.3
Min	342
Max	398
95th Percentile	385
99th Percentile	392

Table D.6: Classification latency statistics per leaf (1000 runs)

Statistic	Time (ms)
Mean	578
Median	576
Std Dev	8.7
Min	558
Max	612
95th Percentile	592
99th Percentile	605

D.6 Model Size Comparison

Table D.7: Model size comparison with other edge AI solutions

Model	FP32 (MB)	INT8 (KB)	Compression
ESPDet-Pico	1.91	478	4.0×
MobileNetV2 (ours)	10.92	2,730	4.0×
ResNet18	44.7	11,175	4.0×
EfficientNet-B0	20.3	5,075	4.0×

D.7 Battery Life Under Different Scenarios

Table D.8: Estimated battery life (3000 mAh battery)

Capture Interval	Avg Current (mA)	Battery Life
1 minute	7.8	16 days
5 minutes	2.3	54 days
10 minutes	1.2	104 days
30 minutes	0.4	312 days

D.8 LoRaWAN Packet Structure Details

D.8.1 Uplink Packet Format

Table D.9: LoRaWAN uplink packet structure

Field	Type	Size (bytes)	Description
Device ID	uint32	4	Unique device identifier
Timestamp	uint32	4	Unix timestamp
Battery Level	uint8	1	Percentage (0-100)
Num Leaves	uint8	1	Number of detected leaves
Disease Code	uint8	1	0=Healthy, 1=Black Rot, etc.
Confidence	uint8	1	Percentage (0-100)
Entropy	uint8	1	Scaled to 0-255
Reserved	-	3	Future use
Total		16	

D.9 Comparison with State-of-the-Art

Table D.10: Comparison with related work on edge AI plant disease detection

System	Platform	Accuracy	Latency	Power
Wang et al. [X]	Raspberry Pi 4	94.2%	250ms	2.5W
Li et al. [X]	Jetson Nano	96.8%	120ms	5W
Chen et al. [X]	STM32H7	91.5%	800ms	0.3W
Ours	ESP32-S3	98.7%	6.3s	0.8W

D.10 Error Analysis

D.10.1 Common Misclassification Patterns

Table D.11: Most common misclassifications

True Class	Predicted Class	Count
Esca	Leaf Blight	12
Leaf Blight	Esca	8
Black Rot	Esca	5
Healthy	Black Rot	3



Quick Links

- [Go to Abstract](#)
- [Go to Table of Contents](#)
- [Go to Introduction](#)
- [Go to Literature Review](#)
- [Go to Edge AI System Design and Implementation](#)
- [Go to Results and Discussion](#)
- [Go to Conclusion and Future Work](#)
- [Go to Bibliography](#)
- [Go to Appendix A: Code Repository](#)
- [Go to Appendix B: Hyperparameters](#)
- [Go to Appendix C: Build Instructions](#)
- [Go to Appendix D: Additional Results](#)